情報処理安全確保支援士試験　科目A-2　中分類26 セキュリティ実装技術・評価　#17

セキュア開発とWebの防御

作り込まない、ブラウザに守らせる、部品と鍵を管理する

各ページの下に、根拠にした資料（IPA の支援士試験シラバス案、IPA「安全なウェブサイトの作り方」、OWASP、RFC 6265・6797・9111、W3C CSP・SRI、OpenSSF SLSA など）を書いた。過去問は IPA 公開の午前 II 。例の場面は Claude が作った

全体像 シラバス案 中分類26 小分類1「アプリケーション開発のセキュリティ」

1 開発の進め方と実装上の対策
SAST・DAST・ファジング、エスケープ・プレースホルダ・CSRF トークン・BOF 対策・ソルトとストレッチング

2 ブラウザの防御機構
同一オリジン・CORS、Secure・HttpOnly・SameSite、HSTS・CSP・SRI・Cache-Control

3 サプライチェーンと実行環境
シークレット管理・依存ライブラリ・署名付きコミット・成果物の署名・SLSA・要塞化

過去の午前 II での出方（7回出題）

テーマ	出た回
cookie の Secure 属性	R1秋・R2・R3秋
HSTS	R3春・R4春・R6春
Cache-Control: no-store	R6春

> XSS・SQL インジェクション・CSRF などの攻撃の手口は #03。ここは「どう作れば防げるか」。

出典：IPA「支援士試験 シラバス（案）Ver.0.1」中分類26 小分類1／IPA 午前 II 各回（このPDFの問題演習を参照）

第1章

セキュア開発の進め方と実装上の対策

見つける検査と、作り込まない書き方。

脆弱性の検査技術

設計（脅威モデリング） → **実装（SAST・SCA）** → **テスト（DAST・ファジング）** → リリース前（ペネトレーションテスト） → 運用（脆弱性情報の監視）

方法	見るもの	得意	苦手
SAST	ソースコード（動かさない）	入力から危険な処理への流れを網羅	誤検知、設定・環境の問題
DAST	動くアプリ（外から攻撃を模す）	実際に通るか、設定の不備	通らない画面、原因の場所
ファジング	異常な入力への反応	クラッシュ・メモリ破壊	論理的な欠陥

早い段階で見つけるほど直すのが安い（**シフトレフト**）。指針は IPA「安全なウェブサイトの作り方」、NIST SSDF。

出典：IPA「支援士試験 シラバス（案）」用語例〔セキュア開発の進め方〕／IPA「ファジング活用の手引き」／NIST SP 800-218

根本的な対策の書き方

SQL インジェクション:プレースホルダ

```
// 危険：文字列連結
"SELECT * FROM users WHERE id='" + id + "'"
// 安全：構造を確定してから値を渡す
ps = conn.prepareStatement(
  "SELECT * FROM users WHERE id = ?");
ps.setString(1, id);
```

XSS:文脈に合ったエスケープ

HTML 本文	< > & を文字参照に
属性値	引用符で囲み " も変換
href	http・https だけ許可

CSRF:推測できないトークン

```
<input type="hidden" name="csrf"
  value="Zx81kQ...（セッションごとの乱数）">
```

パスワードの保存

ソルト
利用者ごとの乱数。レインボーテーブル対策

ストレッチング
計算を重く。Argon2id・scrypt・bcrypt・PBKDF2

> SHA-256 を1回、共通の固定値、可逆な暗号化・Base64 は不可。

出典:IPA「安全なウェブサイトの作り方」1.1・1.5・1.6／OWASP「Password Storage Cheat Sheet」

バッファオーバーフローの対策

スタックのイメージ



対策	しくみ	誰が
長さの確認	strcpy → 長さを指定できる関数、入力長の検査	開発者
メモリ安全な言語	Rust・Java・Go など	開発者
スタックカナリア	戻り先の手前の値の改ざんを検知	コンパイラ
DEP・NX	データ領域のコードを実行させない	OS・CPU
ASLR	メモリ配置をランダムにしアドレスを推測させない	OS

> 緩和策は回避されることもある（ROP など）。根本は長さを確かめること。

出典：IPA「セキュア・プログラミング講座」／IPA「支援士試験 シラバス（案）」用語例「バッファオーバーフロー対策」

第2章

ブラウザが備える防御機構と指示

ヘッダと属性で、ブラウザに守らせる。

cookie の属性（R1秋・R2・R3秋）

```
Set-Cookie: SID=4f9a...; Secure; HttpOnly; SameSite=Lax; Path=/; Max-Age=1800
```

属性	動作	防ぐもの
Secure	スキームが **https のときだけ**送る。値はもたない	平文の通信での盗聴
HttpOnly	JavaScript から読めない	XSS での cookie 盗用
SameSite	他サイト発の要求で送らない（Strict・Lax）	CSRF
Path・Domain	送るパス・ホストを決める	—
Expires・Max-Age	有効期限。なければブラウザ終了で消える	—

過去問の誤りの選択肢：有効期間で無効化＝Expires、JavaScript の読出し禁止＝HttpOnly、パスのプレフィックス＝Path、「Secure=時間」「Secure=ホスト名」は存在しない書き方。

出典：RFC 6265 HTTP State Management Mechanism 4.1.2／IPA 午前 II 令和3年度秋期 問10・令和2年度 問12

同一オリジンポリシーと CORS

オリジン＝スキーム＋ホスト＋ポート

https://app.example.jp と比べて	同一？
https://app.example.jp/x	同一
http://app.example.jp	別（スキーム）
https://api.example.jp	別（ホスト）
https://app.example.jp:8443	別（ポート）

CORS で読取りを許可する

```
Access-Control-Allow-Origin: https://app.example.jp
Access-Control-Allow-Credentials: true
```

「*」や、要求の Origin をそのまま返すのは危険。認証情報つきでは「*」は使えない。

CORS は「読める相手」を広げる仕組み。CSRF の対策ではない。

出典：RFC 6454 The Web Origin Concept／WHATWG「Fetch Standard」CORS protocol

HSTS・CSP・SRI・Cache-Control

指示	書き方の例	効果
HSTS	Strict-Transport-Security: max-age=31536000; includeSubDomains; preload	以降の期間、全て HTTPS で接続（R3春・R4春・R6春）
CSP	Content-Security-Policy: script-src 'self' 'nonce-r4nd0m'; frame-ancestors 'none'	許可外のスクリプトを実行させない、枠表示を禁止
SRI	script src="https://cdn…/lib.js" integrity="sha384-…"	改ざんされた外部ファイルを実行しない
no-store	Cache-Control: no-store	どのキャッシュにも保存させない（R6春 問20）

HSTS の誤りの選択肢：他のオリジンにアクセスさせない＝同一オリジン、EV 証明書の表示、秘密情報を判定できない圧縮、再ハンドシェイク。

no-cache は「保存してよいが毎回確認」、**private** はブラウザだけ保存可。混同に注意。

出典：RFC 6797（HSTS）・RFC 9111（HTTP Caching）／W3C「CSP Level 3」「Subresource Integrity」／IPA 午前 II 令和6年度春期 問13・問20

第3章

ソフトウェアサプライチェーンと実行環境

部品・鍵・ビルドの過程まで守る。

ソースからデプロイまでの守り方



シークレットを push してしまったら：消しても履歴に残る → **直ちに無効化して再発行**、不正利用を確認、シークレットストアへ。

SLSA v1.0	ビルドの段階
L1	来歴（プロヴナンス）がある
L2	ホスト型のビルド環境が署名した来歴
L3	改ざんに強いビルド環境

出典：OpenSSF「SLSA v1.0」／OWASP「Secrets Management Cheat Sheet」／IPA「支援士試験 シラバス（案）」用語例〔ソフトウェアサプライチェーン及び実行環境の保護〕

まとめ：本番での問われ方

検査：SAST＝動かさずコード、DAST＝動かして外から、ファジング＝異常な入力を大量に。

根本対策：SQLi＝プレースホルダ、XSS＝出力時のエスケープ、CSRF＝トークン、BOF＝長さの確認、パスワード＝ソルト＋ストレッチング。

サプライチェーン：シークレットのハードコード禁止、署名付きコミット、成果物の署名、SLSA、要塞化。

cookie：Secure＝https のときだけ送る、HttpOnly＝JS から読めない、SameSite＝他サイト発では送らない。

HSTS：以降の指定期間、全て HTTPS で接続。**no-store：**どのキャッシュにも保存させない。

CSP＝実行してよいスクリプト、**SRI**＝外部ファイルのハッシュ、**CORS**＝読ませてよいオリジン。

出典：IPA 午前 II（このPDFの問題演習）／IPA「支援士試験 シラバス（案）Ver.0.1」